NoSQL & Cloud Firestore

Database design

NIFR - 2023 https://github.com/nicklasdean/firebase

NoSQL & Cloud Firestore

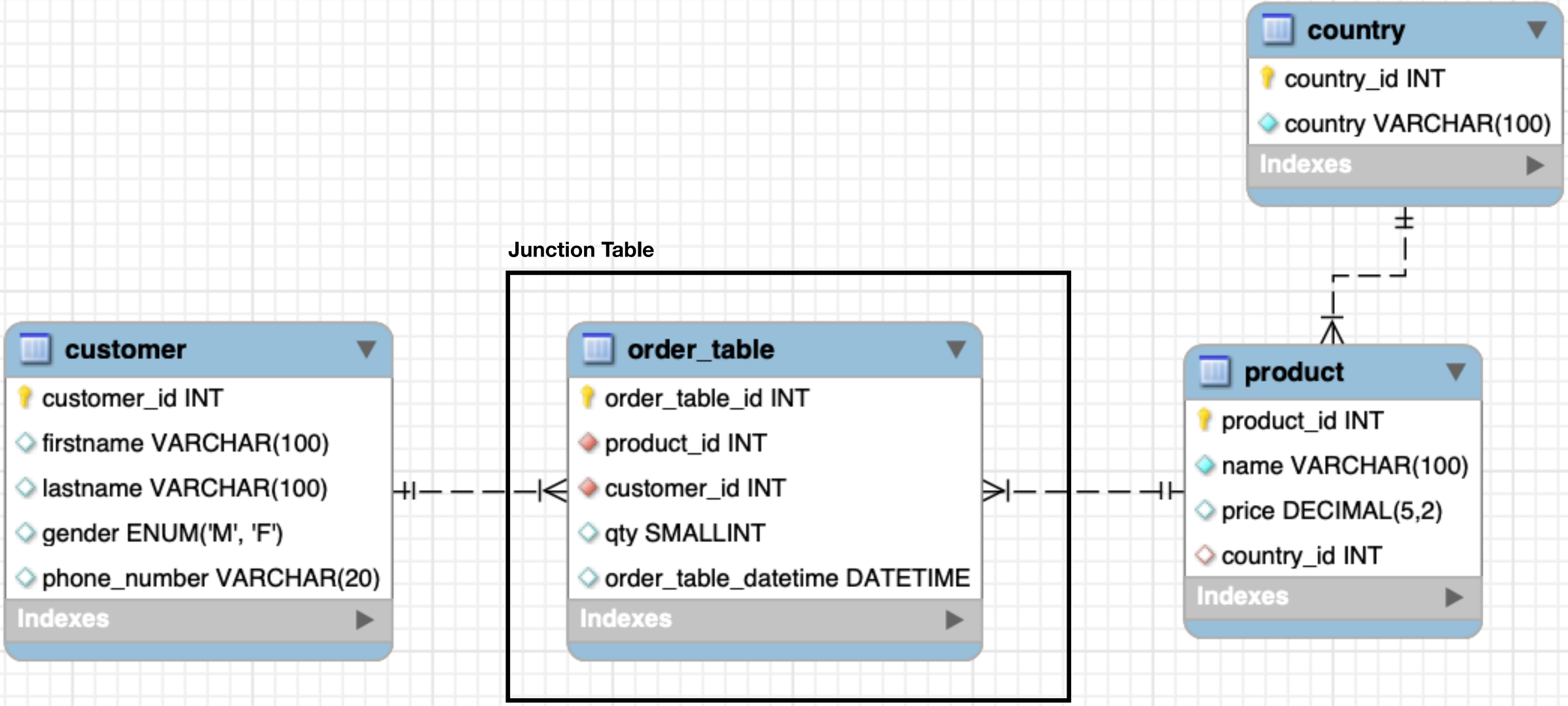
Agenda

- NoSQL - Document oriented databases
- Cloud Firestore: Collection, documents, sub-collections
- Writing Queries to Cloud Firestore
  - Create
  - Read
  - Update
  - Delete

Relationship

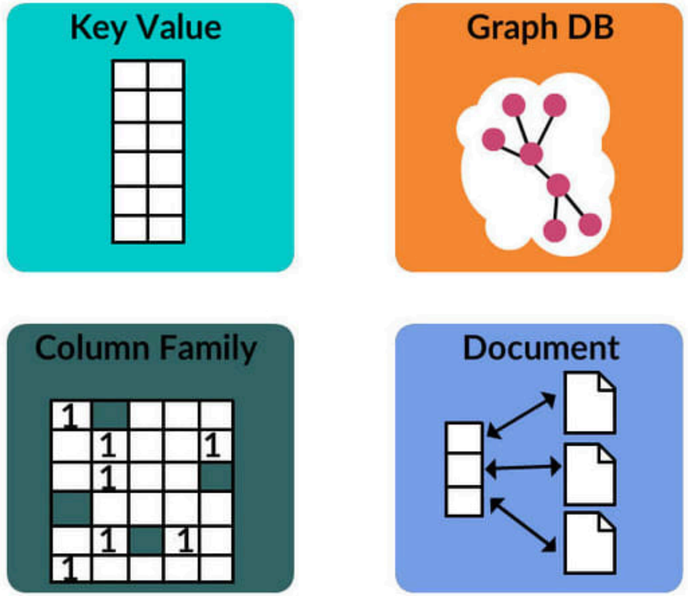
id	employee_name	job	manager	hiredate	salary	commission	department_number
7369	SMITH	CLERK	7902	1980-12-17	800	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-04-02	2975	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250	1400	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450	NULL	10
7788	SCOTT	ANALYST	7566	1987-04-19	3000	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500	0	30
7876	ADAMS	CLERK	7788	1987-05-23	1100	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300	NULL	10

department_number	department_name	location
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON


Junction Table
customer
customer_id INT
firstname VARCHAR(100)
lastname VARCHAR(100)
gender ENUM('M', 'F')
phone_number VARCHAR(20)
Indexes
order_table
order_table_id INT
product_id INT
customer_id INT
qty SMALLINT
order_table_datetime DATETIME
Indexes
product
product_id INT
name VARCHAR(100)
price DECIMAL(5,2)
country_id INT
Indexes
country
country_id INT
country VARCHAR(100)
Indexes

NoSQL

Not SQL or not only SQL



- Alternatives to persistence by relational database management systems (RDBMS)
- Does not structure data as rows and columns in tables
- Solutions to problems within the relational model
- The following weeks will focus on document oriented databases which is the second most popular after RDBMS

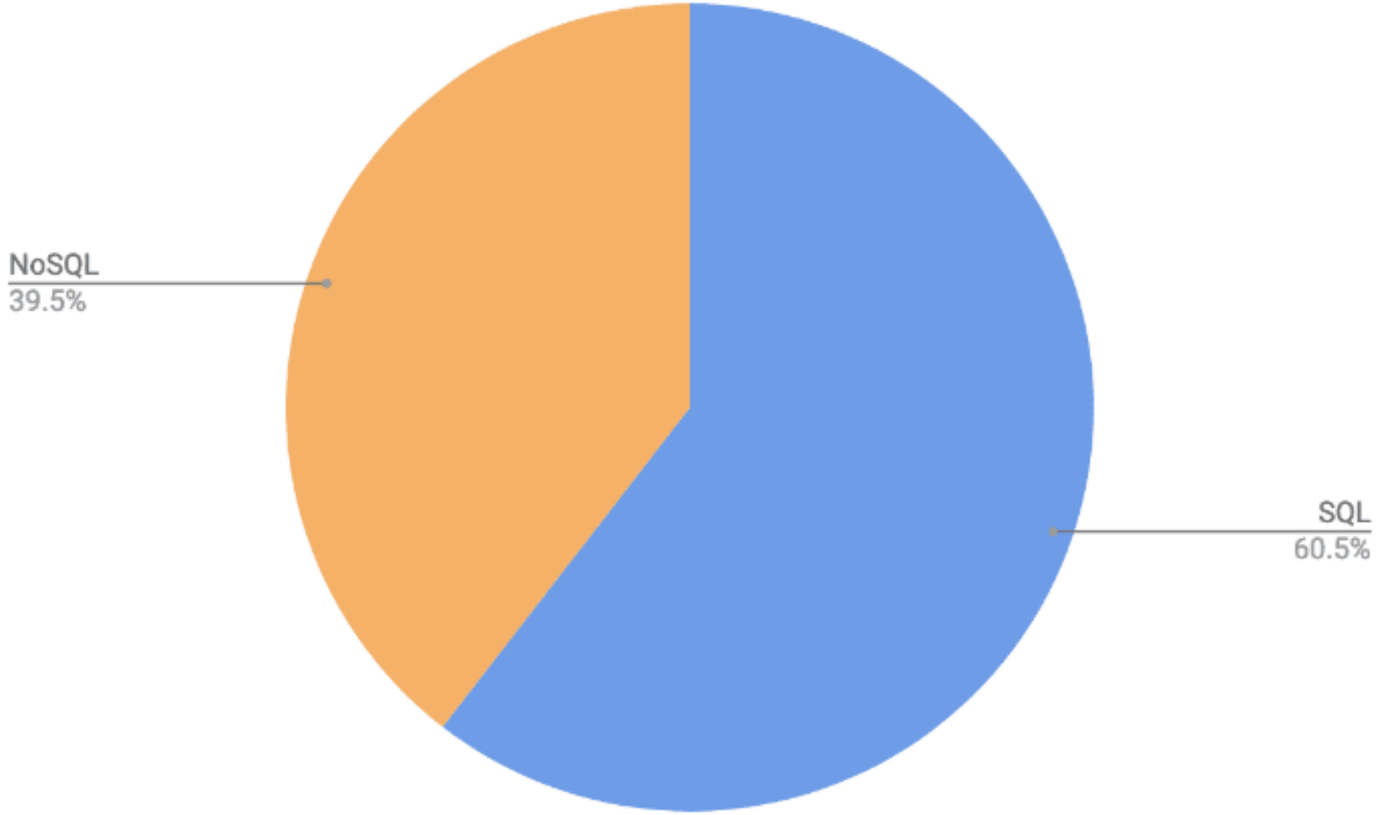
mongoDB

neo4j

GraphQL

redis

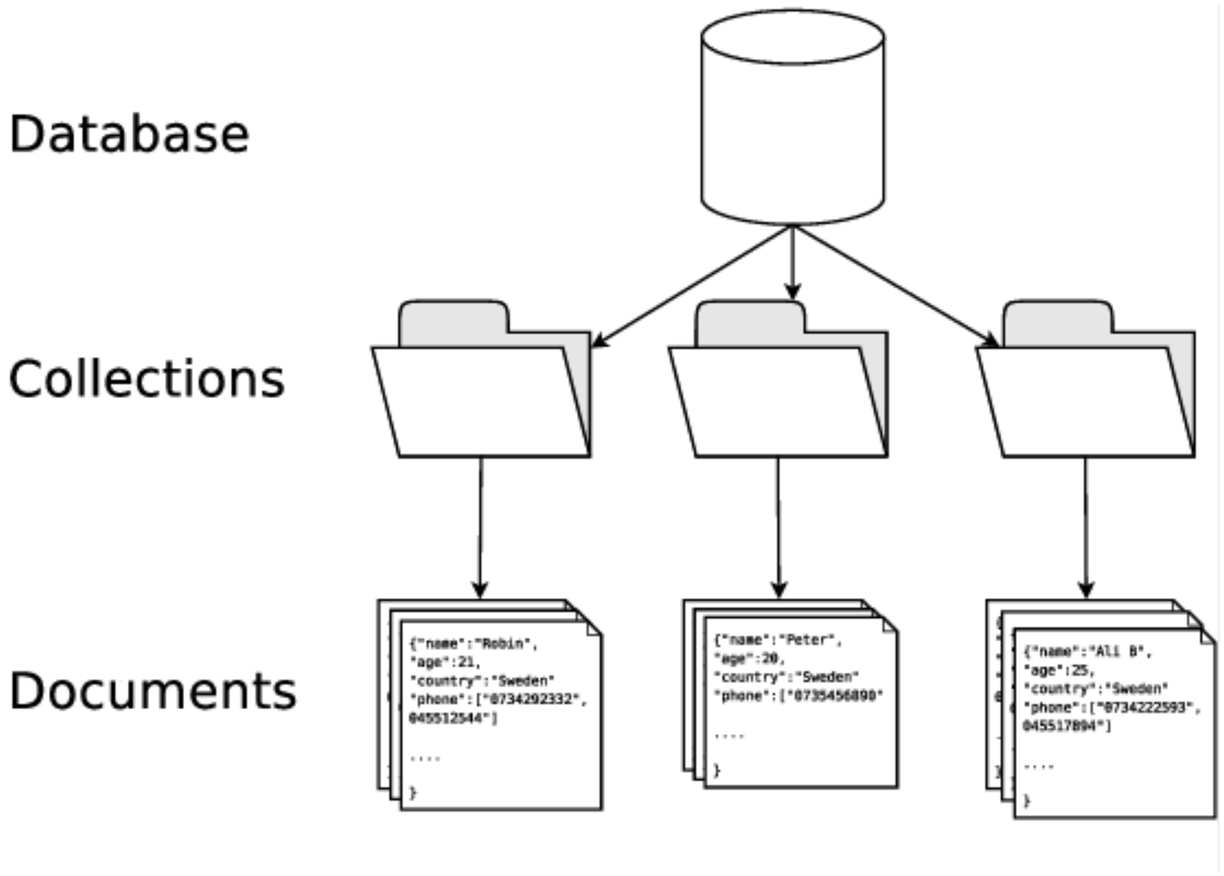
Market share (from a developer survey)

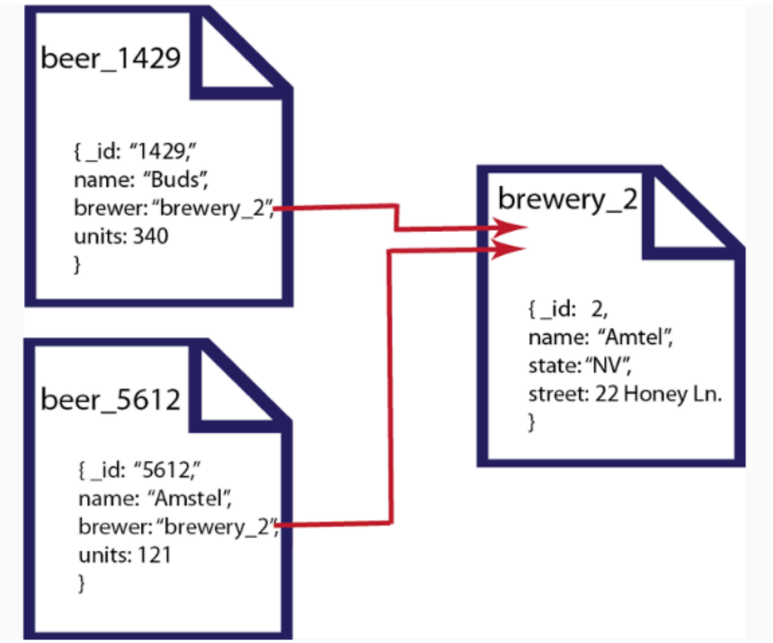


https://scalegrid.io/blog/2019-database-trends-sql-vs-nosql-top-databases-single-vs-multiple-database-use/

NoSQL

Document oriented databases

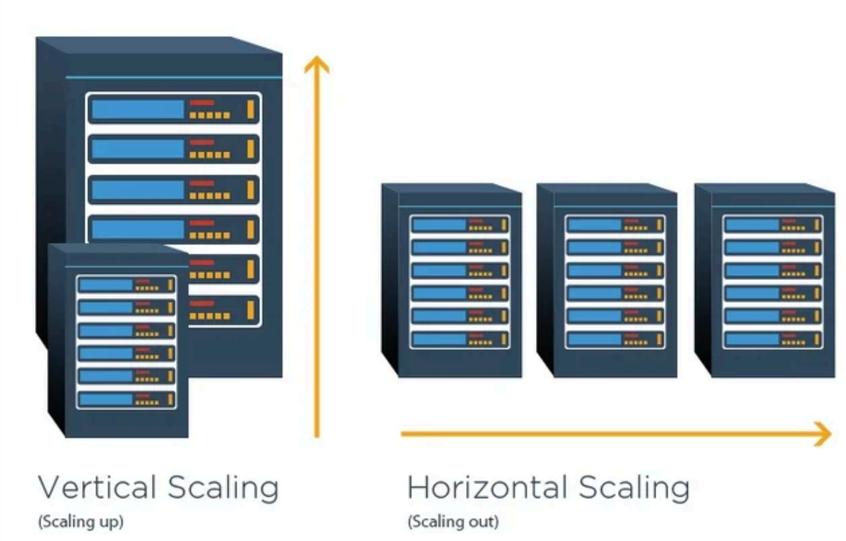




Why

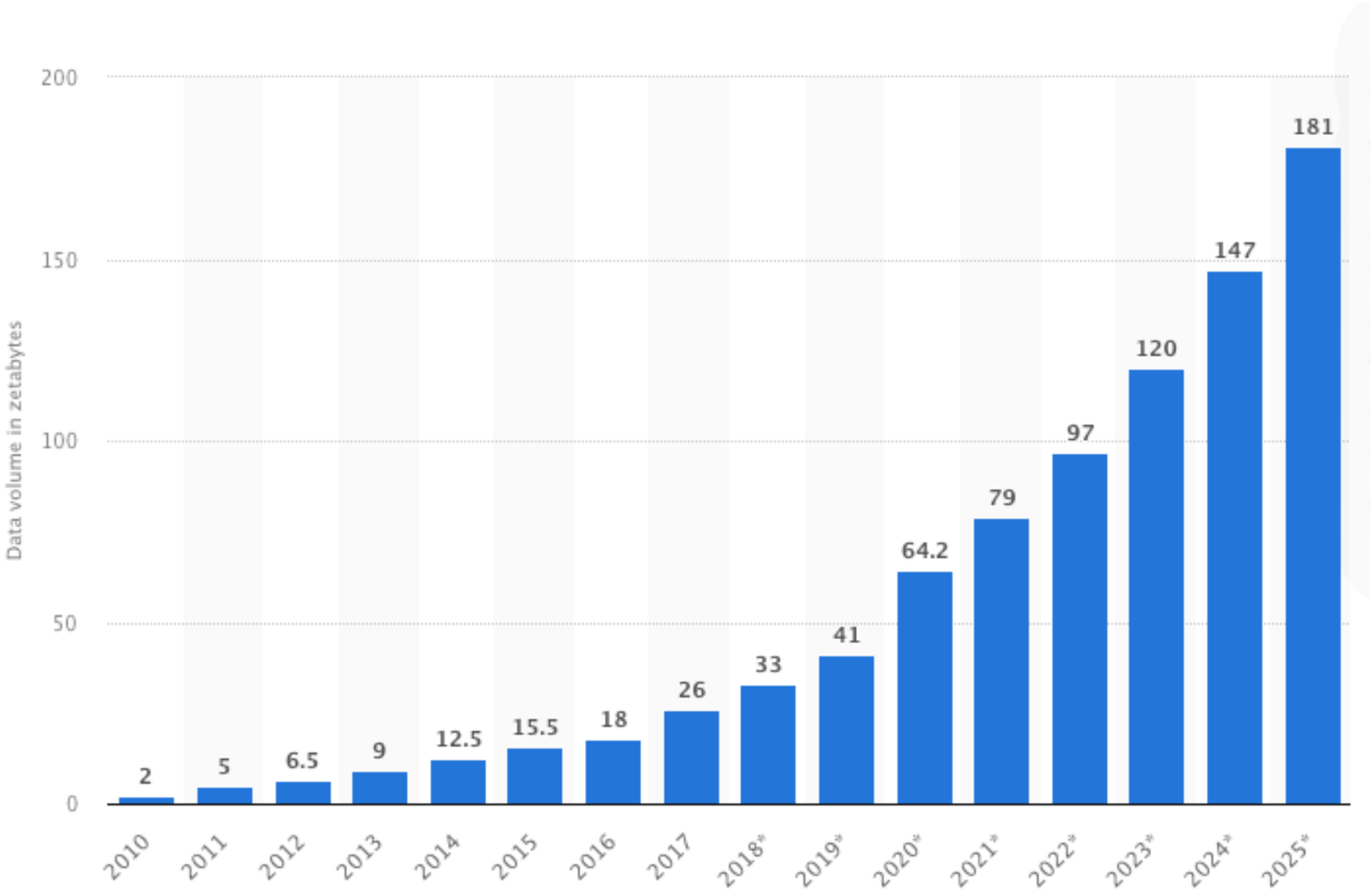
Document oriented databases

- Horizontally scaling
- Can store unstructured & semi-structured data
- Json-like by default
- Flexible scheme: New “columns” can be added easily



Document oriented databases can scale horizontally

BIG DATA



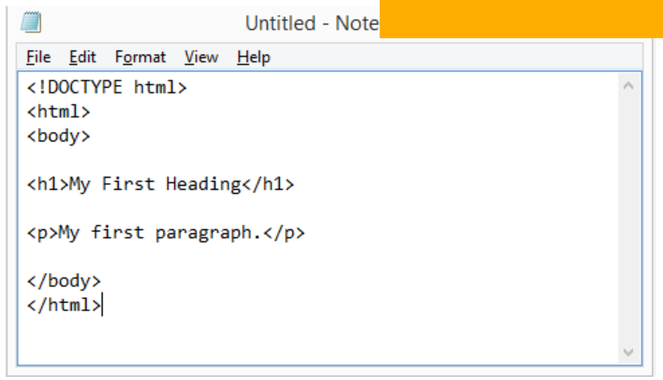
https://www.statista.com/statistics/871513/worldwide-data-created/

Unstructured data

Why NoSQL / Document oriented databases

Social media data

HTML pages

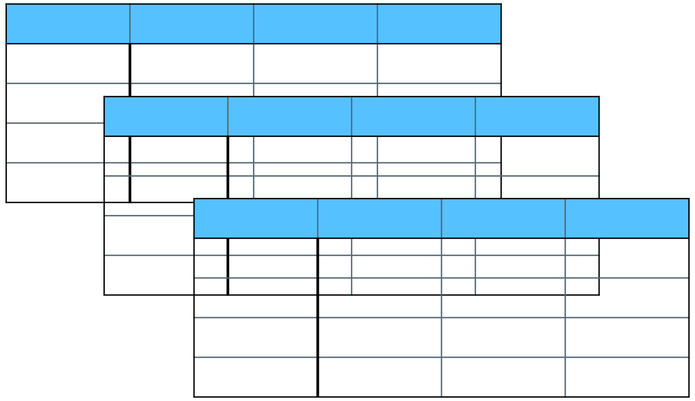


JSON data

RDBMS

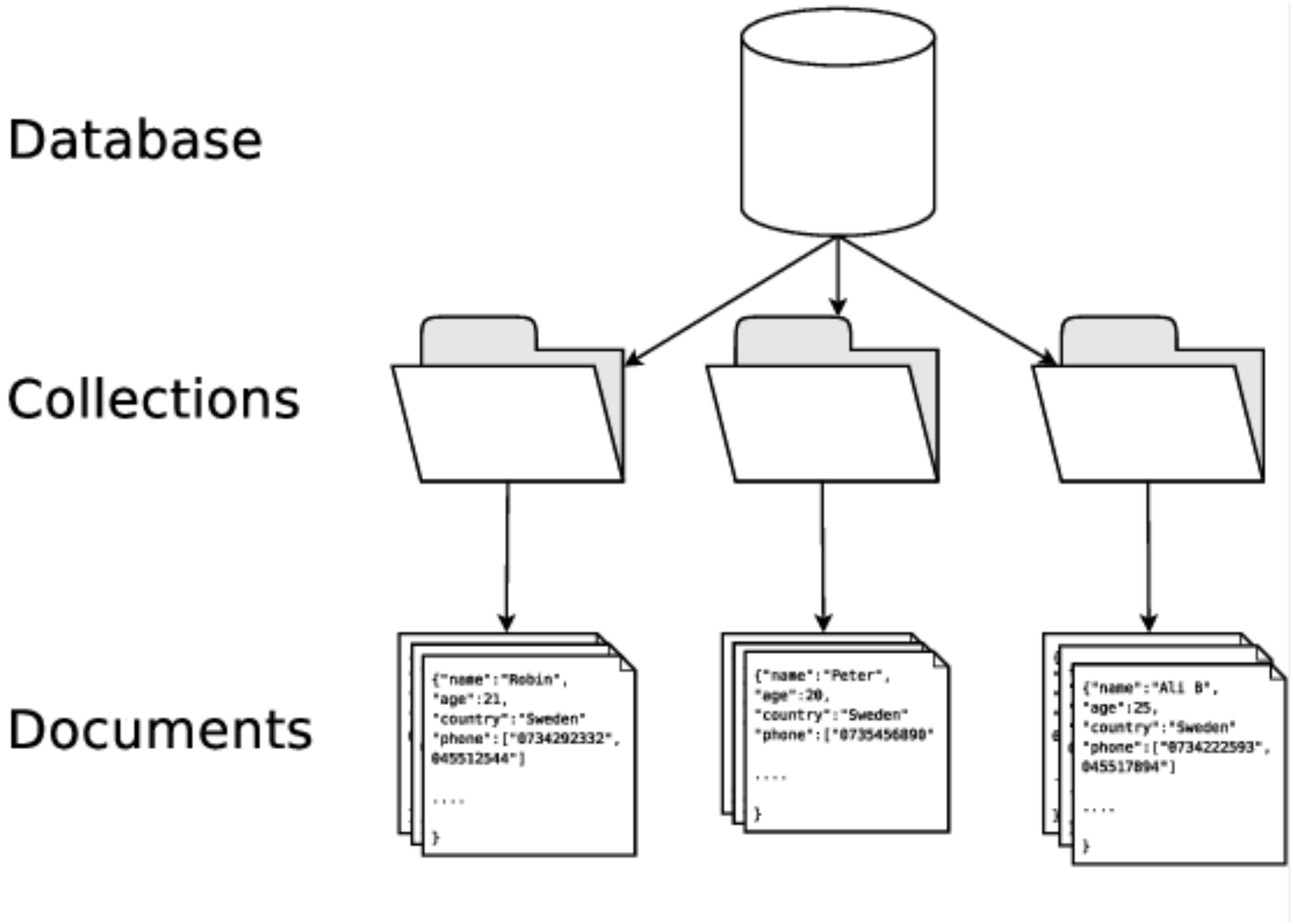
Database / Schema

Tables

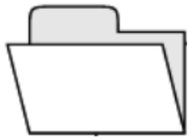


Rows and columns (data)

Document oriented database



Flexible scheme

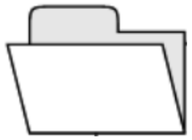



```
{
  "name":"Nicklas",
  "age":32,
  "hobbies":["Dota2","chess","music"]
}
```

```
{
  "name":"Karsten",
  "age":10
}
```

```
{
  "name":"Bob",
  "age":45,
  "hobbies":["Bobsled","judo"],
  "children":["Karsten"]
}
```

Flexible scheme




```
{
  "name":"Nicklas",
  "age":32,
  "hobbies":["Dota2","chess","music"]
}
```

```
{
  "name":"Karsten",
  "age":10,
  "pet":"fish"
}
```

```
{
  "name":"Bob",
  "age":45,
  "hobbies":["Bobsled","judo"],
  "children":["Karsten"]
}
```

Reading & Creating data

CRUD - Create Read Update Delete

Web modular API | Web namespaced API | Swift | Objective-C | Kotlin+KTX Android | More

Use the `setDoc()` method:

```
import { doc, setDoc } from "firebase/firestore";

// Add a new document in collection "cities"
await setDoc(doc(db, "cities", "LA"), {
  name: "Los Angeles",
  state: "CA",
  country: "USA"
});
```

set_document.js

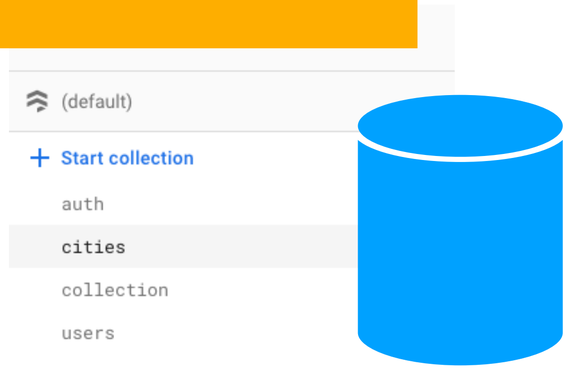
https://firebase.google.com/docs/firestore/manage-data/add-data#web-modular-api_1

Adding data

setDoc Function

setDoc requires 2 parameters

Parameter 1: Collection



Parameter 2: JSON data

```
{
  "name":"Karsten",
  "age":10,
  "pet":"fish"
}
```

Parameter 1: Collection

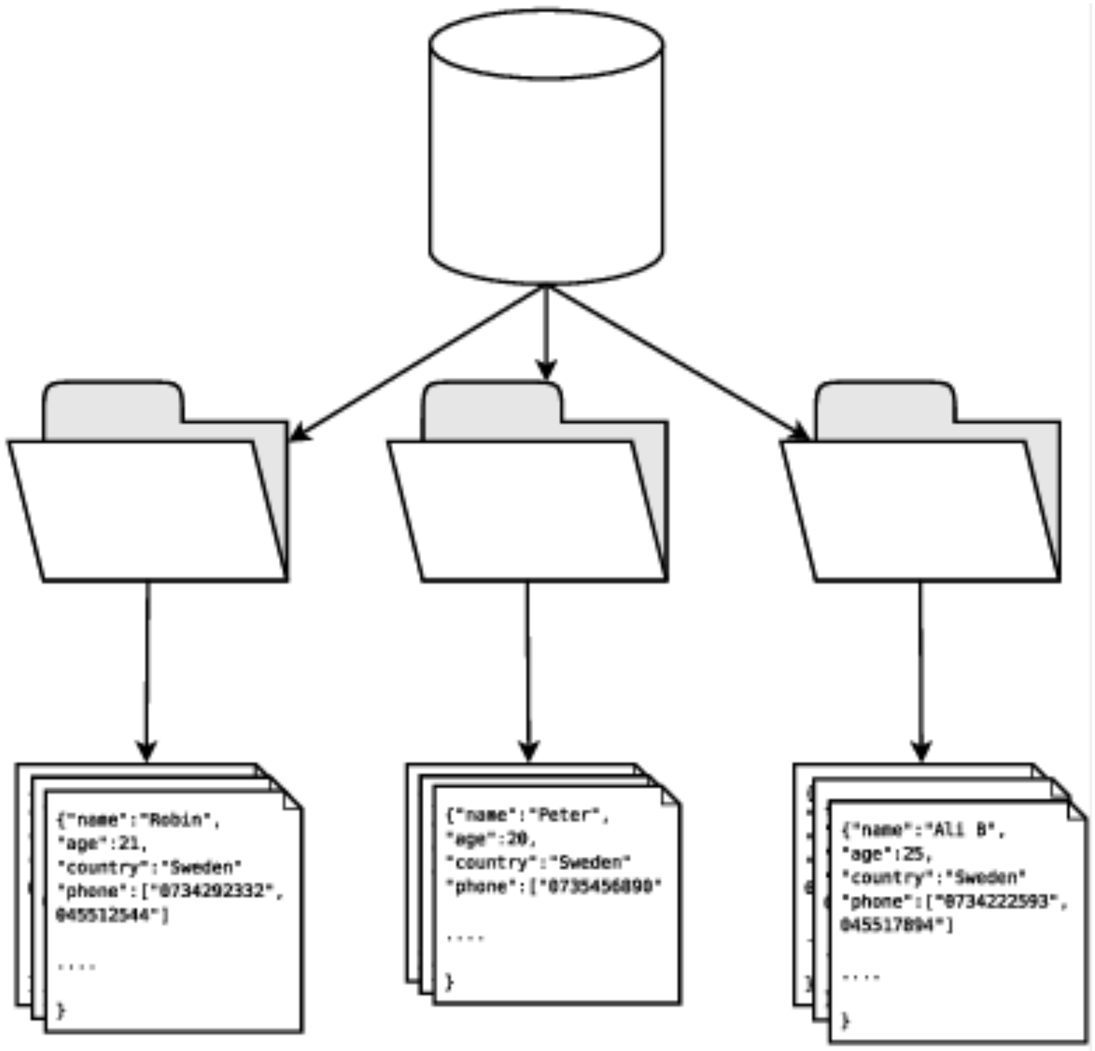
```
const citiesRef = collection(db, "cities");

await setDoc(doc(citiesRef, "SF"), {
    name: "San Francisco",
    state: "CA",
    country: "USA",
    capital: false,
    population: 860000,
    regions: ["west_coast", "norcal"]
});
```

Parameter 2: Data

Database

Collections

Documents



Collections of documents

All fields of all documents are indexed

Example: Adding data

Reading data

Learning objectives

- Retrieving a single document by ID
- Retrieving all documents in a collection
- Basic queries using where( )
- Querying an element within an array
- Aggregation queries (average, sum, count)
- Compound queries (**and, or**)

Retrieving a single documents by ID

Get all documents from collection

Cloud Firestore

- Reference the firebase app
- Get firestore instance from firebase
- Reference the collection we want to fetch data from
- getDocs from the collection
- ForEach -> Iterate each document

Process

Fetching a snapshot of a collection

```
const snapshot = await getDocs(cities);
```

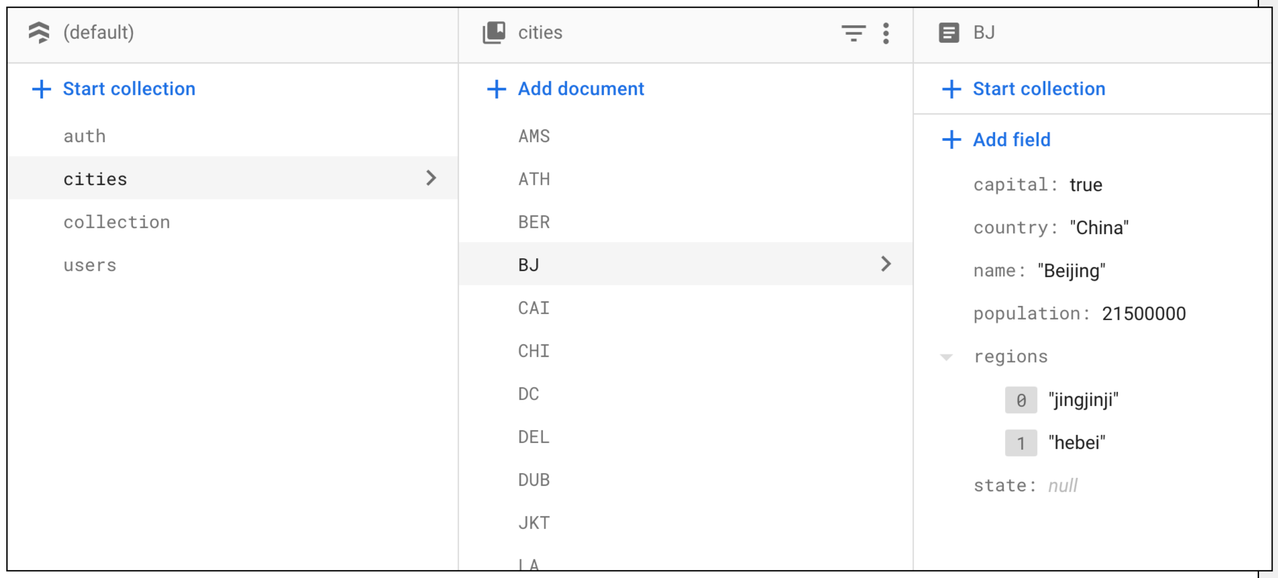
```
snapshot = [“CPH,”LDN”,”DUB”,”ATH”]
```

```
snapshot.forEach()
```

Retrieving all documents of a collection

Prerequisites

Querying data in firebase



My firebase console

```
import { collection, doc, setDoc } from "firebase/firestore";

const citiesRef = collection(db, "cities");

await setDoc(doc(citiesRef, "SF"), {
    name: "San Francisco", state: "CA", country: "USA",
    capital: false, population: 860000,
    regions: ["west_coast", "norcal"] });
await setDoc(doc(citiesRef, "LA"), {
    name: "Los Angeles", state: "CA", country: "USA",
    capital: false, population: 3900000,
    regions: ["west_coast", "socal"] });
await setDoc(doc(citiesRef, "DC"), {
    name: "Washington, D.C.", state: null, country: "USA",
    capital: true, population: 680000,
    regions: ["east_coast"] });
await setDoc(doc(citiesRef, "TOK"), {
    name: "Tokyo", state: null, country: "Japan",
    capital: true, population: 9000000,
    regions: ["kanto", "honshu"] });
await setDoc(doc(citiesRef, "BJ"), {
    name: "Beijing", state: null, country: "China",
    capital: true, population: 21500000,
    regions: ["jingjinji", "hebei"] });
```

example_data.js

https://firebase.google.com/docs/firestore/query-data/get-data

Cloud firestore queries

Where

```
// Create a reference to the cities collection
import { collection, query, where } from "firebase/firestore";
const citiesRef = collection(db, "cities");

// Create a query against the collection.
const q = query(citiesRef, where("state", "==", "CA"));
```

- `<` less than
- `<=` less than or equal to
- `==` equal to
- `>` greater than
- `>=` greater than or equal to
- `!=` not equal to
- `array-contains`
- `array-contains-any`
- `in`
- `not-in`

https://firebase.google.com/docs/firestore/query-data/queries

Dispatching a query and retrieving result

Where

```
// Create a reference to the cities collection
import { collection, query, where } from "firebase/firestore";
const citiesRef = collection(db, "cities");

// Create a query against the collection.
const q = query(citiesRef, where("state", "==", "CA"));
```

```
const querySnapshot = await getDocs(q);
querySnapshot.forEach((doc) => {
  // doc.data() is never undefined for query doc snapshots
  console.log(doc.id, " => ", doc.data());
});
```

- `<` less than
- `<=` less than or equal to
- `==` equal to
- `>` greater than
- `>=` greater than or equal to
- `!=` not equal to
- `array-contains`
- `array-contains-any`
- `in`
- `not-in`

https://firebase.google.com/docs/firestore/query-data/queries